

Student

I've learned foundations, Kimball, SCD, star vs snowflake, lakehouse, and feature stores. Now I want to actually **do it** - model a real business end-to-end.

ChatGPT • AI Assistant

This is your **capstone lab**. We'll design a complete retail sales warehouse for a fictional company **Mango Mart** - from a whiteboard business scenario to runnable DDL.

Every technique from Chapters 1-6 comes together here. The lab is structured as Kimball's **4-step design process**, expanded to 6 practical steps:

1. **Business scenario overview** - meet Mango Mart.
2. **Step 1** - identify the business process.
3. **Step 2** - declare the grain.
4. **Step 3** - identify the dimensions.
5. **Step 4** - identify the facts.
6. **Step 5** - draw the star.
7. **Step 6** - write the DDL.
8. **Lab review & summary**.

CONCEPT CARD

The Kimball 4-step design ties directly to what you'll produce in this lab:

- **Business process** → the operational activity being analyzed.
- **Grain** → what one row in the fact table represents.
- **Dimensions** → the descriptive context around each fact row.
- **Facts** → the numeric measures per row.

Steps 5 (draw) and 6 (DDL) are the practical delivery - turning the design into a runnable schema.

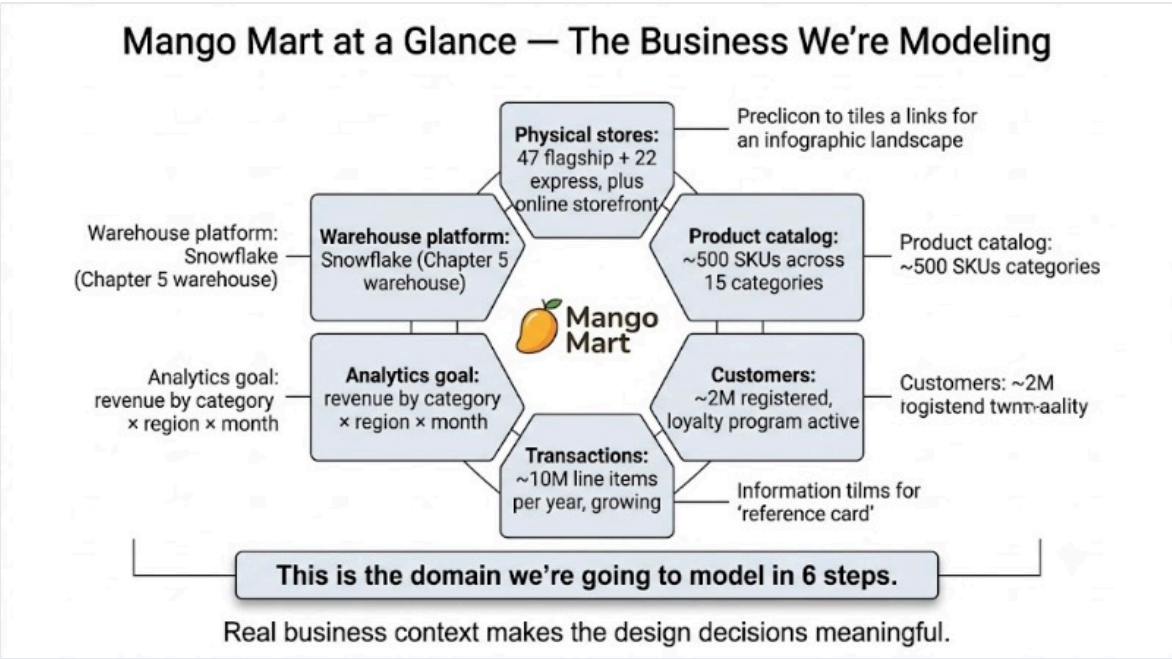
SUMMARY CARD

By the end of this lab you'll have:

- A one-sentence grain declaration for `fact_sales`.
- A prioritized dimension list with attributes.

- A measure list separated into additive / semi-additive / non-additive.
- A star schema diagram.
- Runnable DDL for `fact_sales` + 6 conformed dimensions ready for Lab 2's KPI queries.

[AI Prompt] Warm-up (paste into any AI chatbot): *"Walk me through Ralph Kimball's four-step dimensional design applied to a fictional mid-sized retail company selling 500 products in 50 stores. Return the four decisions in one paragraph each, with reasoning."*



Mango Mart at a Glance — The Business We're Modeling

Lab 1 Milestones — 6 Steps, One Deliverable Each

Step	Deliverable	Chapter Reference
1 — Business Process	One-sentence process statement	Ch 2 (Kimball 4-step)
2 — Grain	One-sentence grain declaration	Ch 2 (grain is sacred)
3 — Dimensions	List of 5-7 dims with attributes	Ch 2 (conformed dims)
4 — Facts	List of measures with additivity	Ch 2 (measure types)
5 — Draw the Star	Star-schema diagram	Ch 4 (star schema)

Step	Deliverable	Chapter Reference
6 — Write the DDL	Runnable CREATE TABLE statements	Ch 3 (SCD), Ch 4 (star)

```
sql – The final deliverable – a preview of what we'll build

-- The lab's final output: a complete star schema for Mango Mart
-- We'll build this piece by piece over the next 6 steps.

-- Central fact table (line-item grain)
CREATE TABLE fact_sales (...);

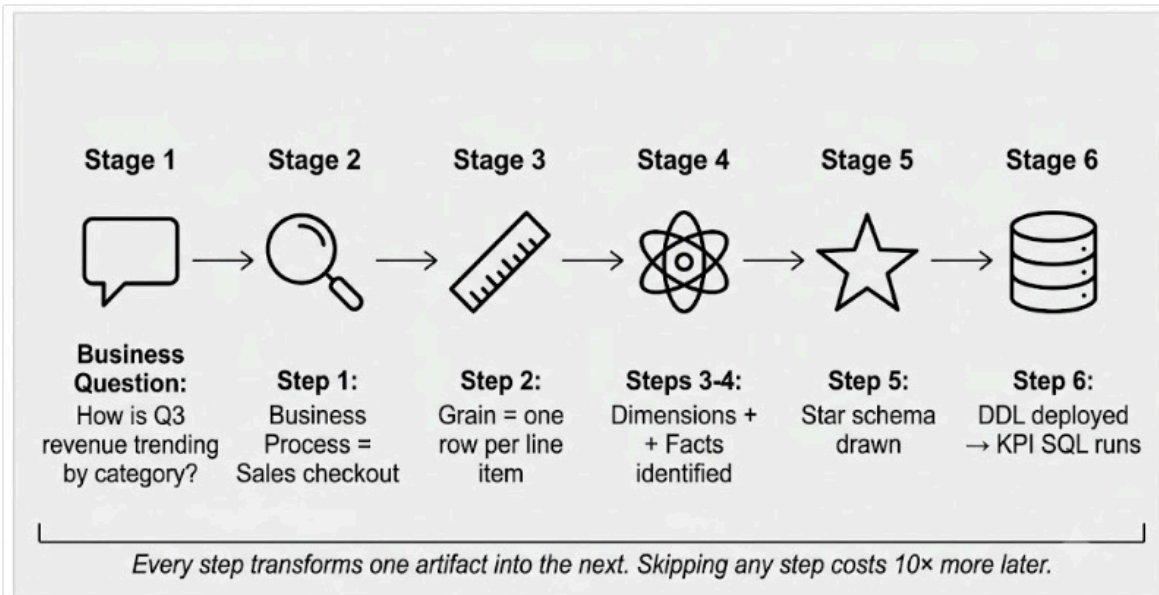
-- Six conformed dimensions
CREATE TABLE dim_date (...);
CREATE TABLE dim_customer (...);    -- SCD Type 2
CREATE TABLE dim_product (...);    -- SCD Type 2
CREATE TABLE dim_store (...);
CREATE TABLE dim_promotion (...);
CREATE TABLE dim_employee (...);    -- SCD Type 2

-- Every technique from Chapters 1-6 will appear in the final DDL.
-- Ready? Let's start with Step 1.
```

LAB OVERVIEW AND BUSINESS SCENARIO



ChatGPT • AI Assistant



The Analytics Journey — From Business Question to Runnable KPI.

The Analytics Journey — From Business Question to Runnable KPI

Student

Set the stage. Who is Mango Mart and what are we trying to help them do?

ChatGPT • AI Assistant

Mango Mart is a mid-sized grocery-and-general-merchandise retailer with a physical footprint plus an online store. The analytics team wants to move from *"we sold something"* dashboards to *"revenue by category × region × month with year-over-year comparison, cohort retention, and promo effectiveness."* They've picked Kimball dimensional modeling; you're the designer.

CONCEPT CARD

The business context in six facts:

1. **Stores:** 47 flagship + 22 express + 1 online = 70 selling locations across 4 regions.
2. **Products:** ~500 SKUs across 15 categories (grocery, dairy, personal care, electronics, apparel, etc.).
3. **Customers:** ~2 million registered users; ~40% enrolled in the loyalty program.
4. **Volume:** ~10 million line items per year; ~2.5M unique transactions.
5. **Promotions:** Weekly campaigns, coupons, and category-level discounts.

6. **Warehouse:** Snowflake, medium-sized team, 15-20 concurrent analysts.

Student

What KPIs do they want to answer?

ChatGPT • AI Assistant

EXAMPLE CARD

The business's top 10 analytical questions:

1. Revenue by category and region per month.
2. Top 10 products by unit sales and revenue.
3. Customer lifetime value by segment.
4. Promo effectiveness (uplift vs baseline).
5. Year-over-year growth by store format.
6. Cohort retention (do new customers come back?).
7. Store performance rankings.
8. Employee sales attribution.
9. Basket analysis (what's bought together).
10. Return / refund rates by category.

Every one of those questions must be answerable from the model we're about to build.

🔗 ANALOGY

A dimensional design lab is like **architecting a house before construction**. You interview the family (business team) about how they'll live, sketch rooms (dimensions) and open spaces (fact tables), draw the plans, then hand blueprints (DDL) to the builders (data engineers).

⚠️ WARNING

Skipping the business-scenario setup is the number-one reason lab designs end up unusable in production. Real teams push back "we don't have time for whiteboarding" - and then rebuild the model 3 months later.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Jumping straight to DDL. You commit to a grain that can't answer half the KPIs; six months later, painful migration.

Correct Way: Spend the first hour on the business scenario. List the top 10 KPIs. THEN design.

EXAM TIP CARD

Interview classic: *"Where do you start when designing a warehouse?"* Winning answer: *"With the business - the top 10 KPIs stakeholders want to answer. Every schema decision (grain, dims, facts) flows from those questions. Design without them is guesswork."*

🔥 EXERCISE

Before proceeding to Step 1, take 5 minutes and list 3 more analytical questions Mango Mart executives might ask that aren't in the top 10 above. Your list will guide the design conversation.

[AI Prompt] Paste this into any AI chatbot: *"For a mid-sized retailer with 70 stores, 500 SKUs, and 2M customers, brainstorm 15 analytical questions their executive team would ask an analytics team. Group by strategic vs operational."*

🔗 PRACTICE QUIZ

Before writing any DDL, what should you have documented?

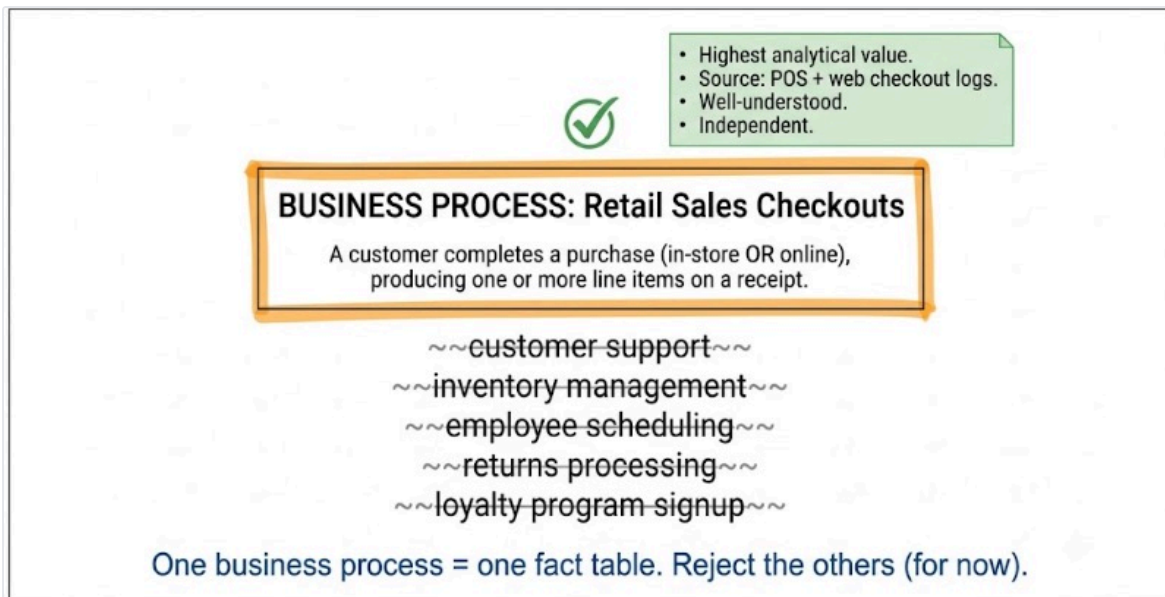
Answer: The top 10-15 analytical questions the business wants to answer.
They drive every design decision.

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

STEP 1 — IDENTIFY THE BUSINESS PROCESS

ChatGPT • AI Assistant



Step 1 Output — The Chosen Business Process on a Whiteboard.

Step 1 Output — The Chosen Business Process on a Whiteboard

Student

What does "identify the business process" actually mean?

ChatGPT • AI Assistant

A **business process** is a discrete operational activity - the thing that happens in the real world and produces data worth analyzing. Sales checkout, order fulfillment, customer support ticket, employee onboarding. **Model one process per fact table.** Never mix processes.

CONCEPT CARD

What counts as a business process:

1. A **specific operational activity** with a clear start and outcome.
2. Produces **measurable events** - transactions, state changes, snapshots.
3. Has **stakeholders** who care about analyzing it.
4. Usually maps to a **source system** or module.
5. Named as a **noun phrase**: "retail sales," "order fulfillment," not "reporting."

Student

How do I pick which process to model first?

ChatGPT • AI Assistant

EXAMPLE CARD

Kimball's guidance - pick a process that is:

1. **Highest analytical value** - most stakeholder questions depend on it.
2. **Feasible** - the source data is trustworthy and available.
3. **Well-understood** - subject-matter experts are accessible.
4. **Independent** - doesn't require modeling 5 other processes first.
5. **Impactful** - winning a report from it changes real decisions.

For Mango Mart, **retail sales checkout** is the obvious choice - most of the top-10 questions center on it.

🔗 ANALOGY

Choosing the business process is like **picking which room to renovate first**. You could tackle the whole house at once - or renovate the kitchen (highest daily impact) and enjoy it while the other rooms wait.

⚠️ WARNING

Don't try to model "the entire business" as one fact table. That produces mixed grain, undefined dims, and unusable reports. One process per fact table.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Naming the process "data" or "reporting" or "everything sales-related." Not specific enough.

Correct Way: "Retail sales checkouts - the moment a customer completes a purchase at any store or online." Specific, verbable, real.

EXAM TIP CARD

Interview classic: *"How do you choose the first business process to model?"*

Winning answer: *"Highest analytical value, feasible source data, well-understood, independent, impactful. For most retailers, sales checkout is the obvious first pick."*

🔥 EXERCISE

Write Mango Mart's business process in **one sentence**. Read it aloud. If it doesn't name a specific operational activity, refine until it does.

[AI Prompt] Paste this into any AI chatbot: *"For each of these industries - banking, hospital, food delivery, streaming service, real-estate - name the 3 highest-value business processes to model as fact tables. Justify each pick."*

Deliverable for Step 1:

> **Business Process: Retail Sales Checkouts** - the operational moment a customer completes a purchase (in-store or online) at Mango Mart, producing one or more line items.

That's Step 1 done.

STEP 2 — DECLARE THE GRAIN

ChatGPT • AI Assistant

GRAIN

One row per line item on a Mango Mart receipt — a single (product × quantity × price) triple at a **specific checkout event**.

Enables (green checkmarks)

- ✓ Top 10 products by revenue
- ✓ Basket analysis (co-purchases)
- ✓ Line-level promo attribution
- ✓ Category × region drilldowns
- ✓ Any aggregation up to daily or monthly

Trades off (yellow warnings)

- ⚠ Larger row count (10M/yr)
- ⚠ More storage than daily-summary grain
- ⚠ Aggregation cost paid at query time

Prefer the finest grain you can afford. Storage is cheap; missing data is expensive.

Step 2 Output — The Grain Declaration and What It Enables.

Step 2 Output — The Grain Declaration and What It Enables

Student

Now - declare the **grain**. Chapter 2 said this was the pivotal decision.

ChatGPT • AI Assistant

Yes. **Grain** is the single most consequential decision in dimensional design. Write it as a **one-sentence declaration** that unambiguously identifies what one row of **fact_sales** represents. Every subsequent choice - dimensions, facts, queries - depends on this.

CONCEPT CARD

Three grain candidates for retail sales:

1. **Line-item grain** - one row per product line on a receipt (finest).
2. **Transaction grain** - one row per receipt (aggregate of lines).
3. **Daily-summary grain** - one row per store per day (coarsest).

The rule from Chapter 2: **prefer the finest grain you can afford**. You can aggregate up; you can't disaggregate down.

Student

Which one for Mango Mart?

ChatGPT • AI Assistant

EXAMPLE CARD

Line-item grain wins. Here's why:

1. It enables **product-level analysis** - top 10 products by revenue.
2. It enables **basket analysis** - "what's bought together with milk?"
3. It enables **promo attribution** at the line - was this line discounted?
4. Storage is cheap; 10M lines per year is trivial for Snowflake.
5. Line-item is easily aggregated to transaction or daily grain for other reports.

🔗 ANALOGY

Grain choice is like **camera resolution**. A 24-megapixel photo can crop to any size. A 100-pixel thumbnail can never blow up. Same for warehouses - always shoot the max resolution the business needs.

⚠️ WARNING

The grain declaration must be **unambiguous** enough that anyone reading it can identify what a fact row represents. Vague grain = mixed data = double-counting.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: "Sales data grain." That's a topic, not a grain.

Correct Way: "One row per line item on a Mango Mart receipt - a single (product × quantity × price) triple at a specific checkout event."

EXAM TIP CARD

Interview classic: *"How do you declare grain?"* Winning answer: *"A one-sentence statement that unambiguously identifies what one row of the fact table represents. It's the pivot on which every downstream decision rests."*

🔥 EXERCISE

Write two possible grain declarations for Mango Mart sales. One at line-item grain, one at transaction grain. Compare which questions each can answer.

[AI Prompt] Paste this into any AI chatbot: *"For a fictional retail sales domain, write 3 possible grain declarations at increasing coarseness (line item, transaction, daily summary). For each, list one KPI it enables and one KPI it cannot answer."*

Deliverable for Step 2:

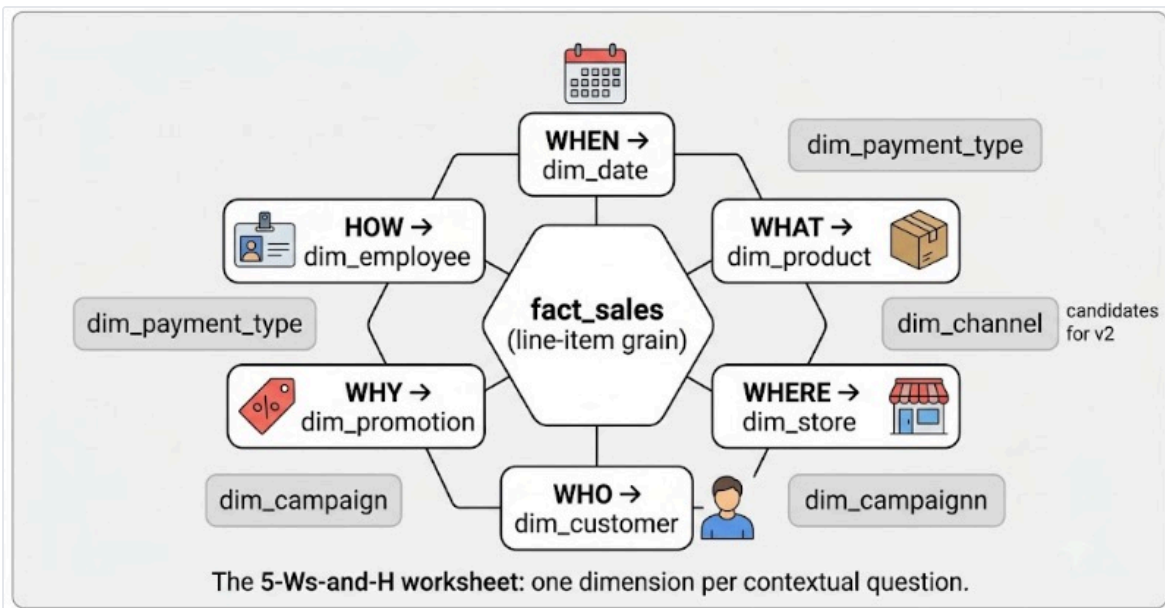
> **Grain: One row per line item on a Mango Mart receipt** - a single (product × quantity × price) triple at a specific checkout event, whether in-store or online.

That's the pivot on which the rest of the design will hinge.

STEP 3 — IDENTIFY THE DIMENSIONS



ChatGPT • AI Assistant



Caption: Step 3 Output — Dimensions from the 5-Whs-and-H Worksheet.

Step 3 Output — Dimensions from the 5-Whs-and-H Worksheet

Student

With grain locked, now identify the **dimensions**.

ChatGPT • AI Assistant

For each line-item fact row, ask: **"What descriptive context would an analyst want to slice this by?"** The answers are your dimensions. Start with the classic 5 Ws: **Who, What, Where, When, Why, How**.

CONCEPT CARD

The 5-Whs-and-H worksheet applied to Mango Mart:

1. **WHEN** did the sale happen? → **`dim\_date`** (also **dim_time_of_day** if hourly matters).
2. **WHO** bought it? → **`dim\_customer`**.
3. **WHAT** did they buy? → **`dim\_product`**.
4. **WHERE** did they buy it? → **`dim\_store`**.
5. **WHY** - was a promotion involved? → **`dim\_promotion`**.
6. **HOW** - who served the customer? → **`dim\_employee`** (for staffed checkouts).

Student

Can I add more dimensions later?

ChatGPT • AI Assistant

EXAMPLE CARD

Additional dimensions to consider (add if the business asks):

1. ``dim_payment_type`` - cash, card, upi, wallet.
2. ``dim_channel`` - in-store, mobile app, website.
3. ``dim_marketing_campaign`` - the attribution touchpoint.
4. ``dim_shipping_method`` - for online orders.
5. ``dim_currency`` - for multi-region operations.

For Mango Mart's first release, we'll stick to the six main dims. Add others as future KPIs demand them.

🔗 ANALOGY

Choosing dimensions is like **choosing which lenses to buy for a camera**. Each lens is a different way to see the same subject. Six well-chosen lenses cover most photos; adding more is a shopping decision.

⚠️ WARNING

Every dimension you add costs ETL effort and BI-team training. Start with the essential 5-7 and add more only when a real KPI needs them.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Adding a `dim_gender`, `dim_age_bracket`, `dim_marital_status` etc. as separate dims. Those are **attributes of `dim\_customer`**, not their own dimensions.

Correct Way: Fold demographic attributes INTO `dim_customer` as columns.

EXAMPLE CARD

For each dimension, list 5-10 attributes analysts will want:

1. **dim_date:** *fulldate, dayofweek, monthname, quarter, year, isweekend, isholiday, fiscal_period.*
2. **dim_customer:** *customerid, name, email, city, region, segment, loyaltytier, signupdate, birthyear.*
3. **dim_product:** *sku, name, category, subcategory, brand, unitcost, is_active.*

4. **dim_store:** *storeid, name, city, region, format (flagship/express/online), openeddate.*
5. **dim_promotion:** *promoid, name, discounttype, campaign, startdate, enddate.*
6. **dim_employee:** *employeeid, name, role, department, hiredate, is_active.*

EXAM TIP CARD

Interview classic: *"How do you identify dimensions?"* Winning answer: *"For each fact row, ask what descriptive context analysts want to slice by - Who, What, Where, When, Why, How. Each answer is a dimension.*

Fold attributes of the same entity into one dim; don't create dims for every column."

🔥 EXERCISE

For Mango Mart's returns process (a hypothetical future fact table), list the 5 dimensions it would need. Notice how many overlap with sales - those are your **conformed dimensions** from Chapter 2.

[AI Prompt] Paste this into any AI chatbot: *"For a Mango Mart-style retailer, propose 6 dimensions with 8-10 attributes each. Use the 5-Ws-and-H framework. Format as a table."*

Deliverable for Step 3:

> **Dimensions:** *dimdate, dimcustomer, dimproduct, dimstore, dimpromotion, dimemployee* - plus SCD Type 2 columns (*effectivefrom, effectiveto, is_current*) on customer, product, and employee.

STEP 4 — IDENTIFY THE FACTS



ChatGPT • AI Assistant

MEASURES FOR fact_sales		
ADDITIVE (sum across ANY dim)	SEMI-ADDITIVE (sum across SOME dims)	NON-ADDITIVE (never sum)
quantity gross_amount discount_amount net_amount tax_amount cost_amount	(none in this transactional fact — semi-additive appears in snapshot facts facts like inventory)	unit_price

DEGENERATE DIMENSIONS (business IDs, no dim table)
order_id order_line_id

Atomic measures only. Derived metrics (margin, ratios) computed at query time.

Step 4 Output — Facts Classified by Additivity.

Step 4 Output — Facts Classified by Additivity

Student

What are the **facts** (measures) for a line-item row?

ChatGPT • AI Assistant

For each dimension key, ask: "**What did we measure at this line-item event?**" The answers are your facts. Then classify each as **additive**, **semi-additive**, or **non-additive** - the fundamental measure typology from Chapter 2.

CONCEPT CARD

The facts for Mango Mart's line-item event:

1. **quantity** - units sold on this line (additive).
2. **unit_price** - price per unit at time of sale (non-additive - never sum unit prices).
3. **gross_amount** - quantity × unit_price (additive).
4. **discount_amount** - dollar value of promo discount on this line (additive).
5. **net_amount** - gross - discount (additive).
6. **tax_amount** - GST/sales tax on this line (additive).
7. **cost_amount** - unit_cost × quantity (additive - enables margin calc).

Student

Any subtle traps here?

ChatGPT • AI Assistant

EXAMPLE CARD

Yes - three classic ones:

1. **Don't store margin directly** - store `net_amount` and `cost_amount` ; let BI compute margin. Stored ratios go stale.
2. **Don't include tax in net_amount** - keep them separate. Tax rules change; separation preserves flexibility.
3. **Don't add derived time-since-something on the fact** - put that in a dimension attribute or compute at query time.

🗨️ ANALOGY

Selecting facts is like **choosing what to weigh on your restaurant's scale**. You weigh raw ingredients (atomic measures). You compute total dish cost from ingredient weights + prices at report time - you don't pre-cook a "dish weight."

⚠️ WARNING

Never put descriptive text on the fact table. `customer_name` belongs in `dim_customer` , not in `fact_sales` . Descriptive strings on facts break Type 2 versioning and bloat the row.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Adding `margin_percent` as a fact. If unit_cost changes, all historical rows show wrong margin.

Correct Way: Store `net_amount` and `cost_amount` atomically; compute margin at query time.

EXAMPLE CARD

Classification of Mango Mart's facts:

1. **Additive:** quantity, grossamount, discountamount, netamount, taxamount, cost_amount.
2. **Semi-additive:** none in this fact (semi-additive would appear in snapshot facts like inventory).
3. **Non-additive:** unit_price.
4. **Degenerate dimensions on the fact:** orderid, orderline_id (identifiers with no separate dim).

EXAM TIP CARD

Interview classic: *"How do you decide what goes in a fact table vs a dimension?"*
Winning answer: *"Facts hold FKs plus numeric measures - that's it. Descriptive context lives in dimensions. Ratios and derived measures are computed at query time from atomic measures."*

🔥 EXERCISE

For each of the 7 facts above, verify: (a) additivity classification, (b) whether it's atomic or derived. Anything derived → question whether it belongs.

[AI Prompt] Paste this into any AI chatbot: *"For a retail fact_sales at line-item grain, propose 8 measures. For each, classify as additive/semi-additive/non-additive, and note whether it's atomic or derived. Flag any that shouldn't be stored."*

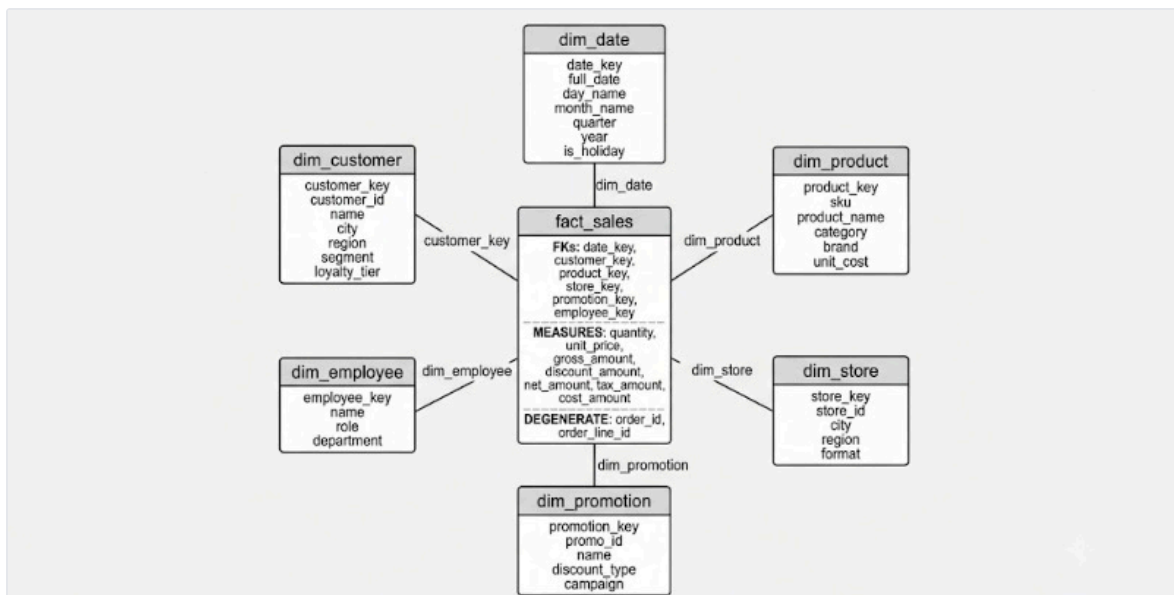
Deliverable for Step 4:

> **Facts (measures):** quantity (additive), unitprice (non-additive), grossamount (additive), discountamount (additive), netamount (additive), taxamount (additive), costamount (additive) - plus degenerate dimensions orderid and orderline_id.

STEP 5 — DRAW THE STAR



ChatGPT • AI Assistant



6 dims + 1 fact + 0 dim-to-dim links = a proper star.

Step 5 Output — The Mango Mart Star Schema.

Step 5 Output — The Mango Mart Star Schema

Student

Now let's draw the **actual star schema** on paper.

ChatGPT • AI Assistant

Time to visualize. In the center place **fact_sales** with its measures listed below.

Around it, arrange the six dimensions with their attribute lists. Draw a straight line from each dimension to the fact, labeling the FK.

If it looks like a star, congratulations - you're doing Kimball right.

CONCEPT CARD

Drawing checklist:

1. **Fact in the center** - with measures listed inside the rectangle.
2. **Six dimensions around it**, evenly spaced.
3. **Each dim has surrogate PK on top**, attributes below.
4. **Straight FK lines** from each dim to the fact.
5. **Star shape** - dimensions do NOT connect to each other.
6. **Degenerate dims** listed inside the fact (no separate table).

What if it's not perfectly symmetrical?

ChatGPT • AI Assistant

EXAMPLE CARD

Real stars are rarely symmetric - that's fine. What matters:

1. **One central fact** - not two.
2. **Dimensions radiate directly to the fact** - no dim-to-dim chains (that would be snowflaking).
3. **Attributes are FLAT inside each dim** - category name in *dimproduct*, *not in a separate dimcategory*.
4. **Layout can be whatever fits the page** - the topology matters more than aesthetics.

🔗 ANALOGY

Drawing the star is like **drafting an org chart**. The layout on paper doesn't affect who reports to whom - but a clear chart helps everyone remember. Same for stars.

⚠️ WARNING

If you find yourself drawing a line between two dimensions, stop. You're on the road to a snowflake. Denormalize the child dim's attributes into the parent.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Drawing `dim_product` connected to a separate `dim_category` connected to `dim_department`. That's a snowflake, not a star.

Correct Way: Category and department attributes flattened INTO `dim_product` as columns.

EXAM TIP CARD

Interview classic: *"What does a star schema look like?"* Winning answer: *"One central fact table with dimensions radiating outward like the points of a star. Every dim connects directly to the fact via a surrogate FK.*

No dim connects to another dim - that would be a snowflake."

🔥 EXERCISE

Sketch Mango Mart's star schema on a whiteboard or paper. Include `fact_sales` in the center with all 6 dimensions around it. Add attributes to each dim.

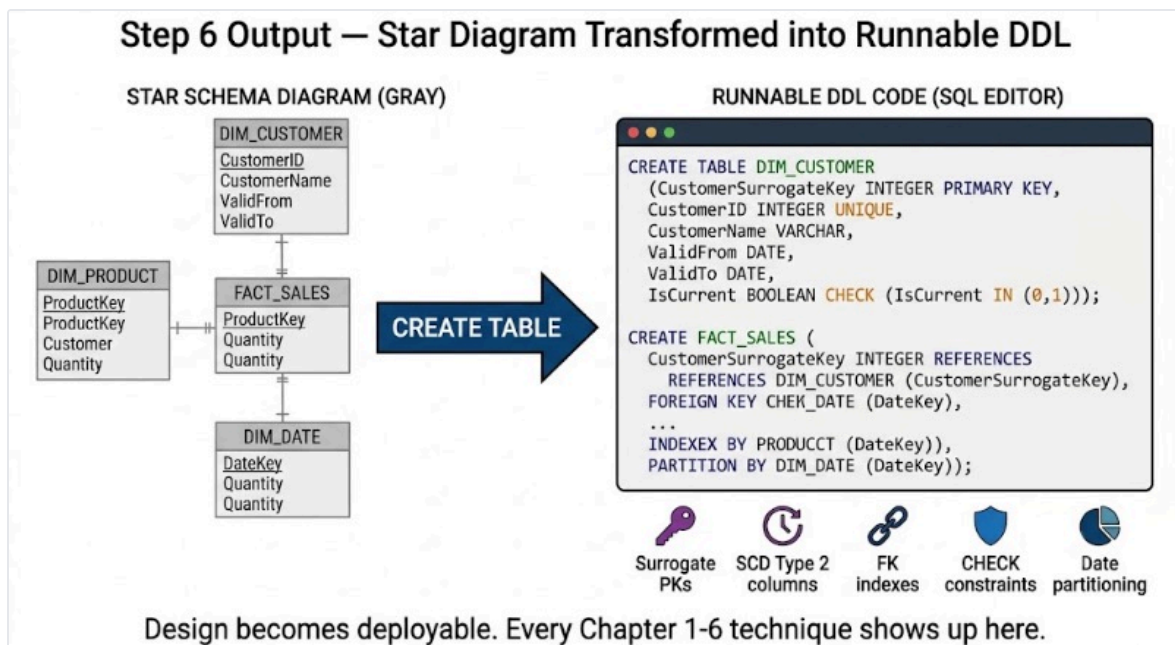
[AI Prompt] Paste this into any AI chatbot: "Given fact_sales at line-item grain with 6 dimensions (dim_date, dim_customer, dim_product, dim_store, dim_promotion, dim_employee), describe the star schema as ASCII art. Show all FKs and 5 attributes per dim."

Deliverable for Step 5:

> A **star schema diagram** with factsales at the center + 6 dimensions arranged radially. No dim-to-dim connections. All hierarchies (category, subcategory, brand) flattened into dim_product.

STEP 6 — WRITE THE DDL

ChatGPT • AI Assistant



Step 6 Output — Star Diagram Transformed into Runnable DDL

sql – Complete Mango Mart star schema – copy, save, run in any SQL warehouse

```
-- =====
-- MANGO MART STAR SCHEMA – LAB 1 DELIVERABLE
-- Grain of fact_sales: one row per receipt line item
-- =====

-- ----- DATE DIMENSION (Type 0, static) -----
CREATE TABLE dim_date (
    date_key          INT          PRIMARY KEY,    -- YYYYMMDD
```

```

full_date      DATE      NOT NULL UNIQUE,
day_of_week    INT,
day_name       VARCHAR(10),
month_number   INT,
month_name     VARCHAR(10),
quarter_name   VARCHAR(10),
year           INT      NOT NULL,
is_weekend     BOOLEAN   DEFAULT FALSE,
is_holiday     BOOLEAN   DEFAULT FALSE,
fiscal_year    INT,
fiscal_quarter INT
);

-- ----- CUSTOMER DIMENSION (SCD Type 2) -----
CREATE TABLE dim_customer (
  customer_key BIGINT      PRIMARY KEY,
  customer_id  VARCHAR(50) NOT NULL,      -- natural key
  full_name    VARCHAR(200),
  email        VARCHAR(200),
  city         VARCHAR(100),
  region       VARCHAR(50),
  country      VARCHAR(80),
  segment      VARCHAR(50),
  loyalty_tier VARCHAR(20),      -- bronze/silver/gold
  signup_date  DATE,
  birth_year   INT,

  -- SCD Type 2
  effective_from DATE      NOT NULL,
  effective_to   DATE,
  is_current     BOOLEAN   NOT NULL DEFAULT TRUE,
  version_number INT      NOT NULL DEFAULT 1
);

CREATE UNIQUE INDEX ux_dim_customer_current
  ON dim_customer(customer_id)
  WHERE is_current = TRUE;

-- ----- PRODUCT DIMENSION (SCD Type 2) -----
CREATE TABLE dim_product (
  product_key BIGINT      PRIMARY KEY,
  sku          VARCHAR(50) NOT NULL,
  product_name VARCHAR(200) NOT NULL,
  category     VARCHAR(100),      -- FLAT: no snowflake
  sub_category VARCHAR(100),
  department   VARCHAR(100),

```

```

brand          VARCHAR(100),
supplier_name  VARCHAR(200),
unit_cost      DECIMAL(12, 2),
is_private_label BOOLEAN,

effective_from DATE          NOT NULL,
effective_to   DATE,
is_current     BOOLEAN      NOT NULL DEFAULT TRUE,
version_number INT          NOT NULL DEFAULT 1
);

CREATE UNIQUE INDEX ux_dim_product_current
  ON dim_product(sku)
  WHERE is_current = TRUE;

-- ----- STORE DIMENSION (Type 1 mostly) -----
CREATE TABLE dim_store (
  store_key      BIGINT      PRIMARY KEY,
  store_id       VARCHAR(20) UNIQUE NOT NULL,
  store_name     VARCHAR(200),
  store_city     VARCHAR(100),
  region        VARCHAR(50),
  country       VARCHAR(80),
  store_format   VARCHAR(20),          --
flagship/express/online
  opened_date    DATE,
  is_active      BOOLEAN      DEFAULT TRUE
);

-- ----- PROMOTION DIMENSION (Type 0) -----
CREATE TABLE dim_promotion (
  promotion_key  BIGINT      PRIMARY KEY,
  promo_id       VARCHAR(30) UNIQUE NOT NULL,
  promo_name     VARCHAR(200),
  discount_type  VARCHAR(20),          -- percent/amount/bogo
  campaign       VARCHAR(100),
  start_date     DATE,
  end_date       DATE
);

-- ----- EMPLOYEE DIMENSION (SCD Type 2) -----
CREATE TABLE dim_employee (
  employee_key   BIGINT      PRIMARY KEY,
  employee_id    VARCHAR(20) NOT NULL,
  full_name      VARCHAR(200),
  role           VARCHAR(50),

```

```

department    VARCHAR(50),
hire_date     DATE,
is_active     BOOLEAN      DEFAULT TRUE,

effective_from DATE      NOT NULL,
effective_to  DATE,
is_current    BOOLEAN      NOT NULL DEFAULT TRUE
);

-- ----- CENTRAL FACT TABLE -----
CREATE TABLE fact_sales (
    sales_key    BIGINT      PRIMARY KEY,

    -- FKs to conformed dimensions (star spokes)
    date_key     INT         NOT NULL
                    REFERENCES dim_date(date_key),
    customer_key BIGINT      NOT NULL
                    REFERENCES dim_customer(customer_key),
    product_key  BIGINT      NOT NULL
                    REFERENCES dim_product(product_key),
    store_key    BIGINT      NOT NULL
                    REFERENCES dim_store(store_key),
    promotion_key BIGINT
                    REFERENCES dim_promotion(promotion_key),
    employee_key BIGINT
                    REFERENCES dim_employee(employee_key),

    -- Degenerate dimensions (business IDs on the fact)
    order_id     VARCHAR(50) NOT NULL,
    order_line_id VARCHAR(50) NOT NULL,

    -- Additive measures
    quantity     INT         NOT NULL CHECK (quantity > 0),
    gross_amount DECIMAL(12, 2) NOT NULL,
    discount_amount DECIMAL(12, 2) NOT NULL DEFAULT 0,
    net_amount   DECIMAL(12, 2) NOT NULL,
    tax_amount   DECIMAL(12, 2) NOT NULL DEFAULT 0,
    cost_amount  DECIMAL(12, 2),

    -- Non-additive
    unit_price   DECIMAL(12, 2) NOT NULL,

    -- Audit
    loaded_ts    TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

```
-- ----- FK INDEXES (crucial for star join speed) -----
CREATE INDEX idx_fs_date      ON fact_sales(date_key);
CREATE INDEX idx_fs_customer ON fact_sales(customer_key);
CREATE INDEX idx_fs_product  ON fact_sales(product_key);
CREATE INDEX idx_fs_store    ON fact_sales(store_key);
CREATE INDEX idx_fs_promotion ON fact_sales(promotion_key);
CREATE INDEX idx_fs_employee ON fact_sales(employee_key);

-- ----- SANITY CHECK: revenue by category, this year -----
SELECT p.category,
       SUM(s.net_amount) AS revenue,
       COUNT(DISTINCT s.order_id) AS orders
FROM   fact_sales   s
JOIN   dim_product  p ON p.product_key = s.product_key
JOIN   dim_date      d ON d.date_key    = s.date_key
WHERE  d.year = 2026
      AND p.is_current = TRUE
GROUP BY p.category
ORDER BY revenue DESC;

-- Design complete. Ready for Lab 2's KPI queries.
```

Student

Now translate the star into **runnable SQL**.

ChatGPT • AI Assistant

Time for the last step - turning the design into a physical schema. We'll write DDL for **fact_sales** and all 6 dimensions, applying every technique from Chapters 1-6: **surrogate keys**, **SCD Type 2** columns on the changing dims, **conformed dim** naming, **degenerate dims** on the fact, and **indexes** on every FK.

CONCEPT CARD

What the DDL below applies:

1. **Surrogate `\_key` PKs** on every dim (Chapter 1).
2. **Natural `\_id` keys** kept as UNIQUE attributes (Chapter 1).
3. **SCD Type 2 columns** - **effective_from**, **effective_to**, **is_current** - on customer, product, employee (Chapter 3).

4. **Flat hierarchies** - category and brand columns inside dim_product; no snowflaking (Chapter 4).
5. **Degenerate dims** - order_id, orderline_id on the fact (Chapter 2).
6. **FK indexes** on the fact for join speed (Chapter 4).
7. **Partitioning by date_key** - planned for large volumes (Chapter 4).

Student

Anything special for the customer dim?

ChatGPT • AI Assistant

EXAMPLE CARD

The customer dim has three SCD Type 2 columns and a UNIQUE index constrained to `is_current = TRUE`. This is the pattern you saw in Chapter 3:

1. Multiple rows can share the same `customer_id` (historical versions).
2. Only ONE row per `customer_id` is `is_current = TRUE`.
3. Historical facts naturally join to their correct surrogate `customer_key`.

🔗 ANALOGY

Writing the DDL is like **turning architectural blueprints into a foundation pour**. Every line matters - a misplaced constraint at this stage becomes a schema migration next month.

⚠️ WARNING

For a real production deploy, add: partitioning strategy, clustering keys (Snowflake) or sort keys (Redshift), permission grants, audit columns (`created_ts` , `created_by`), and comments on every column.

🔥 EXERCISE

Copy the DDL below into a SQL environment. Run it. Then insert 3 sample rows into each dim and 2 sample fact rows. Query with `SELECT * FROM fact_sales JOIN dim_customer USING (customer_key)`. That's your first-touch verification.

[AI Prompt] Paste this into any AI chatbot: *"Take the star schema DDL for Mango Mart (fact_sales + 6 dims) and add: partitioning on date_key, Snowflake clustering keys, and 3 CHECK constraints on the fact for data-quality guards. Explain each addition."*

Deliverable for Step 6:

> Complete, runnable DDL for `fact_sales` + 6 conformed dimensions with SCD Type 2 machinery, surrogate keys, natural keys, degenerate dims, and FK indexes. See the code block below.

LAB REVIEW & SUMMARY

Student

Can you explain what this chapter is about?

ChatGPT • AI Assistant

CHAPTER SUMMARY

You just designed and delivered a complete dimensional model for a mid-sized retailer in six disciplined steps. This lab wove together every technique from Chapters 1-6 into one runnable schema.

- LAB OVERVIEW: Mango Mart - 70 stores, 500 SKUs, 2M customers, 10M line items/year on Snowflake. Top-10 KPIs drive every design decision.
- STEP 1 - BUSINESS PROCESS: Retail sales checkouts - the operational moment a customer completes a purchase. One process per fact table; reject the rest for later releases.
- STEP 2 - GRAIN: One row per line item on a Mango Mart receipt. Finest grain wins - you can always aggregate up, never disaggregate down. Storage is cheap; missing data is expensive.
- STEP 3 - DIMENSIONS: Six dimensions from the 5-Ws-and-H worksheet - *dimdate*, *dimcustomer*, *dimproduct*, *dimstore*, *dimpromotion*, *dimemployee*. Attributes of the same entity fold into one dim.
- STEP 4 - FACTS: *quantity*, *unitprice*, *grossamount*, *discountamount*, *netamount*, *taxamount*, *costamount* - atomic measures only, classified by additivity. Ratios and margins computed at query time.
- STEP 5 - DRAW THE STAR: One central fact + six dimensions in a radial pattern. No dim-to-dim connections. Hierarchies flattened into *dimproduct* and *dimcustomer*.

- STEP 6 - WRITE THE DDL: Complete runnable schema with surrogate PKs, natural key attributes, SCD Type 2 columns on customer/product/employee, FK indexes on all six dimensions of the fact, and a sanity-check query.

Student

Can you break down the next concept?

ChatGPT • AI Assistant

You have a warehouse-ready star schema for Mango Mart. Six dimensions conformed, one fact at line-item grain, SCD Type 2 in place, indexes ready - this is exactly the kind of deliverable a senior data engineer produces on their first week at a real retailer.

In **Lab 2** we're going to *use* this model. You'll write the actual KPI queries for the top-10 analytical questions we listed in Section 7-1 - revenue by category × region × month, cohort retention, promo effectiveness, and more.

Every query joins your `fact_sales` to two or three dimensions and answers a real business question. Time to make the model earn its keep.